

## Week5

### Task1

To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <sys/stat.h>

int main() {
    char path[PATH_MAX];
    char current_dir[PATH_MAX];
    char previous_dir[PATH_MAX] = "";

    printf("Simple Directory Navigation Program\n");
    printf("Commands: cd <path>, pwd, previous, exit\n\n");

    while (1) {
        if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
            perror("getcwd");
            break;
        }
        printf("%s$ ", current_dir);
        if (fgets(path, sizeof(path), stdin) == NULL) {
            break;
        }
        path[strcspn(path, "\n")] = '\0';
        if (strcmp(path, "exit") == 0) {
            printf("Exiting...\n");
            break;
        }
        if (strcmp(path, "pwd") == 0) {
            printf("Current Directory: %s\n", current_dir);
            continue;
        }
        if (strcmp(path, "previous") == 0) {
            if (strlen(previous_dir) == 0) {
                printf("No previous directory available.\n");
                continue;
            }
            char temp[PATH_MAX];
            strcpy(temp, current_dir);
```

```

GNU nano 7.2
    if (chdir(previous_dir) == -1) {
        perror("chdir");
        continue;
    }
    strcpy(previous_dir, temp);

    printf("Changed to previous directory.\n");
    continue;
}
if (strncmp(path, "cd ", 3) == 0) {
    char *new_path = path + 3;
    while (*new_path == ' ') {
        new_path++;
    }
    if (strlen(new_path) == 0) {
        printf("Error: Path cannot be empty.\n");
        continue;
    }

    struct stat st;

    if (stat(new_path, &st) == -1) {
        perror("Invalid path");
        continue;
    }

    if (!S_ISDIR(st.st_mode)) {
        printf("Error: Path is not a directory.\n");
        continue;
    }
    strcpy(previous_dir, current_dir);
    if (chdir(new_path) == -1) {
        perror("chdir");
        continue;
    }

    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd");
        continue;
    }
}

    printf("Directory changed successfully.\n");
    printf("New Working Directory: %s\n", current_dir);
}
else {
    printf("Invalid command.\n");
    printf("Use: cd <path>, pwd, previous, or exit\n");
}
}
return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./directory
Simple Directory Navigation Program
Commands: cd <path>, pwd, previous, exit

/root$ pwd
Current Directory: /root
/root$ cd OSSP
Directory changed successfully.
New Working Directory: /root/OSSP
/root/OSSP$

```

## Task2:

To Identify Built-ins, Create Dispatch Table, Execute In Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 100
#define MAX_ARGS 20
void command_pwd(char **args);
void command_cd(char **args);
void command_help(char **args);
void command_exit(char **args);
typedef struct {
    char *name;
    void (*function)(char **args);
} Command;
Command dispatch_table[] = {
    {"pwd", command_pwd},
    {"cd", command_cd},
    {"help", command_help},
    {"exit", command_exit}
};

int command_count = sizeof(dispatch_table) / sizeof(dispatch_table[0]);
void command_pwd(char **args) {
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("pwd");
    }
}

void command_cd(char **args) {
    if (args[1] == NULL) {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(args[1]) != 0) {
```

```
GNU nano 7.2
        if (chdir(args[1]) != 0) {
            perror("cd");
        }
    }
}

void command_help(char **args) {
    printf("\nAvailable commands:\n");
    printf("  pwd      - Show current directory\n");
    printf("  cd <dir> - Change directory\n");
    printf("  help     - Show available commands\n");
    printf("  exit     - Exit shell\n");
    printf("  Other commands are executed as processes.\n\n");
}

void command_exit(char **args) {
    printf("Exiting shell...\n");
    exit(0);
}

void execute_process(char **args) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return;
    }
    if (pid == 0) {
        if (execvp(args[0], args) == -1) {
            perror("Command execution failed");
            exit(1);
        }
    } else {
        waitpid(pid, NULL, 0);
    }
}

int execute_builtin(char **args) {
    for (int i = 0; i < command_count; i++) {
        if (strcmp(args[0], dispatch_table[i].name) == 0) {
            dispatch_table[i].function(args);
            return 1;
        }
    }
    return 0;
}
```

Output:

```
root@DESKTOP-VMN932K:~# ./built
SIMPLE COMMAND SHELL
myshell> pwd
/root
myshell> help

Available commands:
pwd          - Show current directory
cd <dir>     - Change directory
help         - Show available commands
exit         - Exit shell
Other commands are executed as processes.

myshell> date
Sat Sep 12 03:57:22 UTC 2026
myshell> exit
Exiting shell...
root@DESKTOP-VMN932K:~#
```

Task3:

To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>
#include <errno.h>
#include <limits.h>
#define MAX_PATH 1024
void display_path() {
    char cwd[MAX_PATH];
    if (getcwd(cwd, sizeof(cwd)) == NULL) {
        perror("getcwd");
        return;
    }
    printf("Current Path: %s\n", cwd);
}
void save_state(char *state_file) {
    char cwd[MAX_PATH];

    if (getcwd(cwd, sizeof(cwd)) == NULL) {
        perror("getcwd");
        return;
    }
    FILE *fp = fopen(state_file, "w");
    if (fp == NULL) {
        perror("Unable to save state");
        return;
    }
    fprintf(fp, "%s\n", cwd);
    fclose(fp);
    printf("State saved successfully.\n");
}
void remove_current_directory() {
    char current[MAX_PATH];
    char parent[MAX_PATH];
    if (getcwd(current, sizeof(current)) == NULL) {
        perror("getcwd");
        return;
    }
}
```

```

printf("Directory to remove: %s\n", current);
strcpy(parent, current);

char *last_slash = strrchr(parent, '/');

if (last_slash == NULL || last_slash == parent) {
    printf("Cannot remove root directory.\n");
    return;
}

*last_slash = '\0';
if (chdir(parent) == -1) {
    perror("Unable to change to parent directory");
    return;
}
if (rmdir(current) == -1) {
    perror("Unable to remove directory");
    return;
}
printf("Directory removed successfully.\n");
printf("Now in parent directory: %s\n", parent);
}

void process_exit(char *state_file) {
    printf("\nExit request received.\n");
    save_state(state_file);
    printf("Cleaning up resources...\n");

    printf("Program exiting safely.\n");
    exit(0);
}

int main() {
    char command[100];
    char state_file[] = "shell_state.txt";
    printf("    DIRECTORY MANAGEMENT PROGRAM\n");

    while (1) {
        printf("\nCommands:\n");
        printf("    path      - Display current path\n");
        printf("    remove    - Remove current directory\n");

```

```

        printf("    save      - Save current state\n");
        printf("    exit      - Exit program\n");
        printf("\nEnter command: ");
        if (fgets(command, sizeof(command), stdin) == NULL) {
            process_exit(state_file);
        }
        command[strcspn(command, "\n")] = '\0';
        if (strcmp(command, "path") == 0) {
            display_path();
        }
        else if (strcmp(command, "remove") == 0) {
            remove_current_directory();
        }
        else if (strcmp(command, "save") == 0) {
            save_state(state_file);
        }
        else if (strcmp(command, "exit") == 0) {
            process_exit(state_file);
        }
        else {
            printf("Invalid command.\n");
        }
    }

    return 0;
}

```

Output:

```
root@DESKTOP-VMN932K:~# ./remove
  DIRECTORY MANAGEMENT PROGRAM

Commands:
  path      - Display current path
  remove    - Remove current directory
  save      - Save current state
  exit      - Exit program

Enter command: path
Current Path: /root

Commands:
  path      - Display current path
  remove    - Remove current directory
  save      - Save current state
  exit      - Exit program

Enter command: save
State saved successfully.

Commands:
  path      - Display current path
  remove    - Remove current directory
  save      - Save current state
  exit      - Exit program

Enter command: exit

Exit request received.
State saved successfully.
Cleaning up resources...
Program exiting safely.
root@DESKTOP-VMN932K:~#
```

#### Task4:

To Parse Export Syntax, Update Environment Variables,  
Validate Names, Handle Existing Variables, Export to Child  
Processes, Test Functionality

Source code:

```
1 12
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 200
#define MAX_NAME 100
#define MAX_VALUE 100
int valid_name(const char *name)
{
    int i;
    if (name == NULL || name[0] == '\0')
        return 0;
    if (!isalpha((unsigned char)name[0]) && name[0] != '_')
        return 0;
    for (i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
    }
    return 1;
}
void export_variable(char *input)
{
    char *argument;
    char *equal;
    char name[MAX_NAME];
    char value[MAX_VALUE];
    argument = input + 6;
    while (*argument == ' ' || *argument == '\t')
        argument++;
    if (*argument == '\0')
    {
        printf("export: missing argument\n");
        return;
    }
    equal = strchr(argument, '=');
}
```

```

    if (equal == NULL)
    {
        printf("export: expected NAME=value\n");
        return;
    }
    int name_length = equal - argument;
    if (name_length <= 0 || name_length >= MAX_NAME)
    {
        printf("export: invalid variable name\n");
        return;
    }
    strncpy(name, argument, name_length);
    name[name_length] = '\0';
    if (!valid_name(name))
    {
        printf("export: '%s' is not a valid variable name\n", name);
        return;
    }
    strncpy(value, equal + 1, MAX_VALUE - 1);
    value[MAX_VALUE - 1] = '\0';
    if (setenv(name, value, 1) == -1)
    {
        perror("setenv");
        return;
    }

    printf("Exported: %s=%s\n", name, value);
}

void show_variable(char *name)
{
    char *value = getenv(name);
    if (value != NULL)
        printf("%s=%s\n", name, value);
    else
        printf("%s is not set\n", name);
}

void test_child_process()
{
    pid_t pid = fork();
    if (pid < 0)

```

```

    if (equal == NULL)
    {
        printf("export: expected NAME=value\n");
        return;
    }
    int name_length = equal - argument;
    if (name_length <= 0 || name_length >= MAX_NAME)
    {
        printf("export: invalid variable name\n");
        return;
    }
    strncpy(name, argument, name_length);
    name[name_length] = '\0';
    if (!valid_name(name))
    {
        printf("export: '%s' is not a valid variable name\n", name);
        return;
    }
    strncpy(value, equal + 1, MAX_VALUE - 1);
    value[MAX_VALUE - 1] = '\0';
    if (setenv(name, value, 1) == -1)
    {
        perror("setenv");
        return;
    }

    printf("Exported: %s=%s\n", name, value);
}

void show_variable(char *name)
{
    char *value = getenv(name);
    if (value != NULL)
        printf("%s=%s\n", name, value);
    else
        printf("%s is not set\n", name);
}

void test_child_process()
{
    pid_t pid = fork();
    if (pid < 0)

```

```

    {
        perror("fork");
        return;
    }
    if (pid == 0)
    {
        char *value = getenv("MYVAR");

        if (value != NULL)
            printf("Child process: MYVAR=%s\n", value);
        else
            printf("Child process: MYVAR not found\n");

        exit(0);
    }
    else
    {
        waitpid(pid, NULL, 0);
    }
}

int main()
{
    char input[MAX_INPUT];
    printf("      EXPORT COMMAND SHELL\n");

    printf("Commands:\n");
    printf("  export NAME=value\n");
    printf("  get NAME\n");
    printf("  test\n");
    printf("  exit\n\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0)

```

```

            break;
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0)
            continue;
        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }
        if (strncmp(input, "export", 6) == 0 &&
            (input[6] == ' ' || input[6] == '\t' || input[6] == '\0'))
        {
            export_variable(input);
        }
        else if (strncmp(input, "get ", 4) == 0)
        {
            char *name = input + 4;
            if (valid_name(name))
                show_variable(name);
            else
                printf("Invalid variable name\n");
        }
        else if (strcmp(input, "test") == 0)
        {
            test_child_process();
        }
        else
        {
            printf("Invalid command.\n");
        }
    }

    return 0;
}

```

Output:

```
myshell> export MYVAR=Hello
Exported: MYVAR=Hello
myshell> get MYVAR
MYVAR=Hello
myshell> test
Child process: MYVAR=Hello
myshell> exir
Invalid command.
myshell> exit
Exiting shell...
root@DESKTOP-VMN932K:~#
```